Catalan numbers and the theoretical limits of morphosyntactic variation

Tommaso Mattiuzzi

Goethe University Frankfurt

August 2026*

Abstract

The number of possible binary trees is a function of the number of leaves in the tree. The growth of this function coincides with the series of Catalan numbers, a numerical sequence with many applications in the domain of enumerative combinatorics. As Collins (2022) recognizes, this numerical sequence specifies a theoretical limit on possible ways a string can be parsed via binary Merge. In this paper, I demonstrate that the nanosyntactic Lexicalization Algorithm (Starke 2018, De Clercq et al. 2025) bounds the number of possible outputs of derivations building and externalizing structure in a single Workspace exactly to the Catalan numbers. I then discuss how this result enables a mathematical characterization of the power of a generative grammar building observable linguistic objects from a given selection of input elements. By determining the space of possibilities open in an abstract derivational system, it becomes possible to quantitatively study the role of the independent factors that determine how languages fill this space. From this perspective, the Catalan numbers emerge as fundamental to the understanding of morphosyntactic variation as the result of the interaction of the First and the Third factor (Chomsky 2005).

---

1 Introduction

In enumerative combinatorics, Catalan numbers count several families of objects, including rooted binary trees with a fixed number of leaves (equivalently, fully parenthesized expressions) (Stan-

*This is a preliminary draft. Any feedback or suggestions are warmly welcomed!

ley 2015). In this paper, building on an earlier insight by Collins (2022), I demonstrate that under a nanosyntactic theory of structure building and externalization, the Catalan numbers emerge as a necessary component of the theoretical limit on morphosyntactic variability. Specifically, the number of trees that can be built and externalized by the nanosyntactic Lexicalization Algorithm (Starke 2018, De Clercq et al. 2025) in k cycles within a single derivational Workspace is C_k. On this basis, the total possibilities for derivations involving multiple Workspaces can be computed combinatorially through sums and products of Catalan numbers.

The theoretical relevance of this result lies in its enabling power. Its most immediate effect is to pave the way for the mathematical characterization of the expressive power of syntactic derivations in general as a function of the number of input elements and the hierarchical relations among them. This in turn explicitly specifies the space of possibilities of what any given I-language *could* do, making it possible to systematically compare this to what known languages actually do.

A striking property of the theoretical space of possibilities is its large size: as the Catalan numbers grow exponentially, the space of possible output trees or possible lexicalization patterns for a single combination of atoms reaches the order of 10^4 after 10 cycles and 10^9 after 20 (see below). Actually attested crosslinguistic variation, on the other hand, tends to cluster around few highly frequent patters for a given type of structure, with other possible options exhibited by a rapidly decreasing number of languages, as is the case e.g. for the typological frequency of word-order possibilities in the nominal domain (Cinque 2005, 2025, Abels & Neeleman 2012, Martin et al. 2024). By combining an explicit characterization of the space of possibilities and empirical data about how languages fill this space, it becomes possible to quantitatively assess how domain-general, Third-factor (Chomsky 2005) constraints rein in the expressive power of the derivational system in its instantiations in individual I-languages. Observable linguistic data must result from how the abstract derivational system is put to use within the boundaries set by universal physical and cognitive limitations, (possibly) constraints that are specific to the language faculty as such, and information that is specific to a given individual (I-)language. A systematic study of this complex interaction shaping how the linguistic generative capacity is put to use requires a characterization of the expressive power of syntax in the void. This paper contributes to approaching this characterization.

In what follows, I start by introducing Catalan numbers as applied to grammatical theory by Collins (2022). I then move to a discussion of the issues that emerge once considering the relation between Catalan numbers and structure-building derivations in section 3. In section 4, I argue that these issues can be addressed by adopting a more global perspective that encompasses both structure-building and externalization. I introduce the basic building blocks of the nanosyntactic Lexicalization Algorithm (4.1), and then demonstrate how the number of configurations that the algorithm can produce in a single Workspace grows with the series of Catalan numbers. In 5 I discuss the limits of this claim and identify two directions for future work.

2 Catalan numbers and Parsing

Collins (2022) shows that when parsing a sequence of words in a syntactic structure, assuming binary Merge constraints the number of possible parse trees according to the sequence of Catalan numbers. For any string of n words, the number of possible parses in a binary tree is given by the Catalan number C_m, where $m = n - 1$ is the number of branching nodes:

(1)
$$C_m = \frac{1}{m+1}\binom{2m}{m} = \frac{2m!}{(m+1)!\,m!}$$

Equivalently, the number of ways a parser assuming binary Merge can analyze a string of n words is:

(2)
$$C_{n-1} = \frac{(2n-2)!}{(n)!\,(n-1)!}$$

As an illustration, (3) shows the possibilities for a string of 4 items, and Table 1 shows how the number of possibilities grows for increasingly longer sequences (here, from a 1-word to a 9-word sequence).

(3) String: *4 3 2 1*

a. [4 [3 [2 1]]]

b. [[4 3] [2 1]]

c. [[4 [3 2]] 1]

d. [[[4 3] 2] 1]

e. [4 [[3 2] 1]]

Nr. (words)	1	2	3	4	5	6	7	8	9	…
Nr. (binary parse trees)	1	1	2	5	14	42	132	429	1430	…

Table 1: Number of valid binary parses for a sequence of n words.

As Collins points out, any n-ary Merge where $n > 2$ yields (many) more possibilities, which highlights the relevance of Catalan numbers for syntactic theory as giving mathematical substance to the claim that binary Merge is the most parsimonious operation that yields hierarchical structure, as pointed out at least since (Kayne 1984, Haegeman 1994).

Collins's argument points to the Catalan numbers as a mathematical characterization of something fundamental about a structure-building system that employs binary Merge: Given some input, the formula provides the number of representations that can be built by such a system. Now, to what extent does this generalize beyond processing? After all, Narrow Syntax is a computational system that from an array of atomic elements builds hierarchical representations where such elements are combined in a strictly binary fashion. Can the Catalan numbers be used to characterize the generative power of derivations based on binary Merge in general?

As will detail in the next section, I will pursue a specific version of this question, namely: Can the Catalan numbers be used to characterize the number of possible ways to build and externalize linguistic objects corresponding to one and the same meaning?

3 Catalan numbers and structure-building

The question I introduced reverses the perspective with respect to the parsing scenario discussed by Collins (2022). There, an input string is given by definition, and the parser knows that it must build trees where constituency does not conflict with linear adjacency, and where the number of leaves in the binary trees must coincide with the number of items in the string. In such context, the sequence of Catalan numbers specifies a theoretical limit on the number of possible hypotheses for a parser that has binary Merge and no cues on the (selectional/constituency) relations among the elements in the string, which would contribute to restrict the number of options to be

entertained to converge on a syntactic representation. What happens when we shift the perspective to a the process of building linguistic structures when the input is not a linear string, but a selection of linguistic objects – whatever the theory allows as atoms of syntactic computation – that correspond to some meaning to be encoded?

The obvious issue related to the relation between constituency and compositionality. If we disregard the content and interpretation of the items involved and we only focus on their number n, the number of possible geometries is given by the Catalan number C_{n-1}. For 4 items, we therefore get the same $C_3 = 5$ geometries illustrated in (3), as shown in (4).

(4) a. [w [x [y z]]]

b. [[w x] [y z]]

c. [[w [x y]] z]

d. [[[w x] y] z]

e. [w [[x y] z]]

Indeed, each of these four geometries are plausible trees for some intended meaning, as illustrated in (5).

(5) a. [Sammy [bought [a car]]]

b. [[extremely versatile] [offensive midfielder]]

c. [[The [new fridge]] broke]

d. [[[Very loud] music] plays]

e. [That [[unbearably narcissistic] professor]]

The problem is that for one and the same selection of input atoms – and for one and the same meaning to be encoded – only one tree will feature suitable constituency relations, assuming that each atom receives a fixed interpretation at externalization.[1] What the reasoning above gives us it the total number of possible trees a binary structure-building derivation can produce for just *any* selection of n items, that is, if we disregard the identity of the items. But once the input items

[1]This is essentially the logic exploited by a Free-Merge perspective, whereby all possibilities outlined above are indeed defined, and those that don't have a well-formed representation at the semantic interface are filtered out. From this perspective, all cases in which well-formed semantic representations can be assigned to multiple distinct possible output trees, neither the syntactic nor the semantic system are sufficient to uniquely determine the meaning of the linguistic expression generated.

and the meaning to be encoded are identified, compositionality dictates specific constituency relations. As shown in (6), there are indeed $C_{4-1} = 5$ possible geometries for trees that respect the linear adjacency of items in *extremely versatile offensive midfielder*, but only one of them (6b) has the desired constituency relations among its items.

(6) <midfielder, offensive, versatile, extremely>

a. *[extremely [versatile [offensive midfielder]]]

b. [[extremely versatile] [offensive midfielder]]

c. *[[extremely [versatile offensive]] midfielder]

d. *[[[extremely versatile] offensive] midfielder]

e. *[extremely [[versatile offensive] midfielder]]

In other words, a tension emerges when trying to connect the abstract example discussed above to derivations involving specific linguistic objects. The abstract geometries of binary trees counted by the Catalan numbers defines a space of possibilities, but nothing else being added only one of these trees will be the target one for some intended meaning. In the remainder of the paper, I will argue that this issue does not arise once we reformulate our initial question about the significance of Catalan numbers. Note that while the general properties of structure-building derivations are expected to be universal, a given composition of linguistic objects – in (6), two adjectives modifying a noun, the highest-scoping of which combined with a degree modifier – may obviously differ in its surface realization in different languages. In what follows, my claim will be that the space of possibilities indicated by the Catalan numbers defines the space of this crosslinguistic variation.

From this shifted perspective, the question becomes whether the series of Catalan numbers characterize the boundary of the generative system building linguistic objects involving a given number of atoms and *corresponding to one and the same meaning*. To be clear, the space of possibilities to be characterized no longer exclusively concerns abstract structure-building derivations in Narrow Syntax, but the whole procedure of building the relevant structures and externalizing them. As I will show below, the structure building and externalization procedure represented by the nanosyntactic Lexicalization Algorithm (Starke 2018, De Clercq et al. 2025) predicts exactly the number of possibilities specified by the Catalan numbers to be derivable in a single

derivational Workspace.

Before illustrating this point, however, I will take a short detour. The reformulation of the issue stresses the role of the externalization component as a source of morphosyntactic variation (and its limits). While the perspective that this is the *only* source of variation has been expressed in different forms in the literature (Boeckx 2011, 2016, Starke 2011), it would be fair to object that this is not the only conceivable approach. Assuming the structure-building procedure to be universal, the other available source of crosslinguistic differences concerns the input of this procedure, as in any framework that assumes the input of narrow syntax to correspond to language-specific bundles of functional features.

To the extent that this has a bearing on the size of the space of variation, I will compare the two options from the perspective of the number of possibilities they define, before turning back to the main point of the paper.

3.1 What are the atoms of computation?

From the perspective of narrow syntax, two options are compatible with standard assumptions on the modularity of Grammar: the input of syntactic computations has either the form of bundles of features, or of individual features. The first case covers both late-insertion models of morphosyntax like Distributed Morphology (Halle & Marantz 1993, 1994, Bobaljik 2017) as well as lexicalist models that assume syntax to operate on words, to the extent that all is visible to the syntactic computation in the latter case are the syntactic features that words carry (Chomsky 1995).

The discussion in the previous sections indicates that the Catalan numbers define the number of possibilities for a numeration of elements, without any restriction on the nature of these elements, so this conclusion remains true regardless of what one assumes to be the input of syntax. However, the question pursued here is whether the Catalan numbers can be identified as a limit on morphosyntactic variation. How does the issue of the input of syntax affect the number of total possible outputs?

I will take the conservative assumption of a functional sequence *fseq* uniquely determining an input sequence for the derivation.[2] If the input of syntax is in the form of atomic features, these

[2]Without a *fseq*, a set of n input atoms may be fed to the derivation in $n!$ different orders, determining a higher

features themselves will be ordered by a *fseq*. This means that there is no possible variation in the input, so if the derivational procedure that operates on this input is bound by the Catalan numbers, the total number of possibilities will also be, as I show in the next sections for the nanosyntactic Lexicalization Algorithm.

If instead the input has the form of bundles of features, there are multiple ways in which such features could be bundled together. This is obviously not the case when considering specific I-languages, since in that case the pre-syntactic bundles are a constant by definition. However, the issue I am discussing is that of the theoretical limit on crosslinguistic variability, and this requires considering the possible forms of bundles as a source of crosslinguistic differences. So how many possible ways are there for any given I-language to bundle n features?

To be clear, a definitive answer to this question hinges on the factors constraining bundling, plausibly including semantic constraints as well as domain-general ('Third-Factor') constraints e.g. on the size and composition of bundles, since the latter affect the computational behavior of the objects fed to syntax, and hence the cognitive load connected to derivations that operate on them. For present purposes, the relevant perspective is a purely combinatorial one: Any constraints along the lines I mentioned would, if anything, act on a space of *a priori* possible combinations by restricting it.

To ensure the generality and tractability of the problem in the context of this work, I will make another conservative assumption, namely that for each bundle in the input, there is exactly one feature determining its place in the *fseq*. For simplicity, let's assume that this is the function of categorial features, like v, T, Num, D, *etc.*. For a total n features and k bundles thereof, this means that there must be k such categorial features. Since by present assumptions these categorial features cannot be combined or rearranged, variability is restricted to the assignment of the remaining $n-k$ features to the k bundles. Assuming independence in how each of these remaining features is assigned to a bundle, the total number of possibilities is k^{n-k}, since for each of these $n-k$ features there are k possible choices of a bundle.[3] But if there are k^{n-k} ways to bundle n features in k bundles, there are k^{n-k} different inputs compliant with one and the same *fseq*. Since we already know that the number of possible outputs for a sequence of k elements is

number of possible outputs, as detailed in Appendix B.

[3]For instance, for $n = 5$ total features and $k = 3$ bundles, each of the remaining $n - k = 2$ features can 'choose' among three bundles, so the total number of possibilities is 3 X 3 = 9.

the Catalan number C_{k-1}, the total number of objects that can be built via binary Merge will be $k^{n-k} \cdot C_{k-1}$.

Now, this reasoning assumes the number of bundles k and the identity of categorial features to be fixed. However, nothing independently restricts the number of bundles that are defined for n features to some constant k. So for n features, there might be interlinguistic variability both in how they are bundled and in the number of bundles – that is, both in the number of k features among them and in how the remaining $n - k$ features are assigned to the k bundles. This means that the total number of possibilities defined under pre-syntactic bundling of n features is given by the sum of all possibilities for all possible values of k, as in (7).[4]

(7)
$$Tot_{bundlesin}(n) = \sum_{k=1}^{n} k^{n-k} \cdot C_{k-1}$$

As a visual reference, Figure 1 shows how the number of possibilities grows compared to the baseline represented by the sequence of Catalan numbers for a small array of values of n. To be sure, what I characterized is the number of possibilities in completely abstract terms. The relevant point for the present discussion is that if the derivational system is such that the number of its outputs grows with the Catalan numbers, assuming pre-syntactic feature bundles determines a faster growth in the size of the space of possibilities, which rapidly gets order of magnitudes higher.

4 Catalan numbers and Externalization

Moving beyond the abstract comparisons drawn above, actually observed variation in terms of linguistic outputs will depend on how the output of syntax is externalized. The discussion so far also left open the issue raised in section 3 about the tension between compositionality and variability in constituency: Once a *fseq* of input atoms is assumed and for a given target meaning, variability in the configurations that can be derived via binary Merge is undesired.

The goal of this section is to show that the tension is resolved if we connect these two points

[4]Lifting the conservative assumption made here about the predetermined identity of categorial features, the total number of possibilities grows by a factor determined by the binomial coefficient $\binom{n}{k}$, which gives the number of ways to choose k items from a set of n – in this context, the number of possible choices of k categorial features, one per bundle.

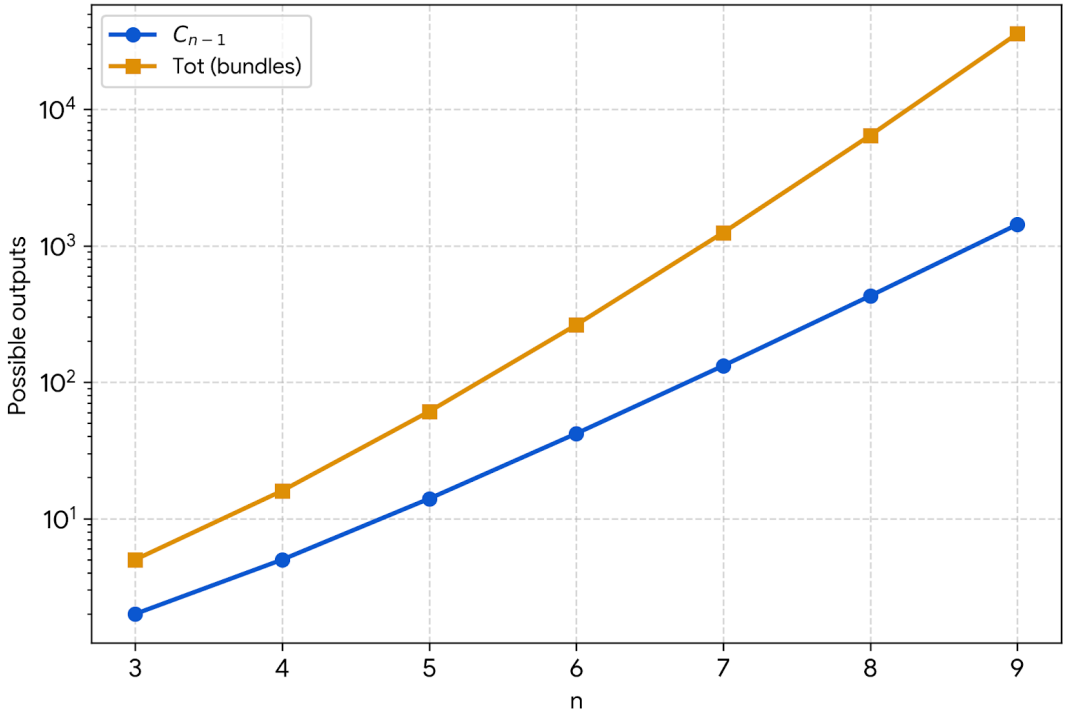


Figure 1: Log-scaled growth rate of the number of possible output trees of a derivation taking n features as input either in the form of atomic features or as bundles, assuming that the input is always ordered by a *fseq*.

by adopting a nanosyntactic perspective on morphosyntactic derivations. I will show that for one and the same selection of features – broadly speaking, for one and the same meaning – the series of Catalan numbers coincides with the growth of the number of possible configurations in which those features can be combined in a single derivational Workspace via the nanosyntactic Lexicalization Algorithm (Starke 2018, De Clercq et al. 2025), which specifies the operations building syntactic structure and licensing its externalization.

4.1 Nanosyntax and the Lexicalization Algorithm

As a stepping stone for the argument to be developed in the next section, let us review the core properties of nanosyntactic derivations. In keeping with the aim of this paper, I will focus exclusively on the relevant aspects of the system, without discussing their empirical predictions. The Reader is referred to De Clercq et al. (2025), Caha (2022), and Baunaz & Lander (2018) for

more detailed presentations of the theory and of the growing body of work based on it.

At the heart of the system is a derivational procedure that builds syntactic structure and licenses it by identifying its possible externalization. Each time a new element is merged to the content of some active derivational Workspace, a proper lexicalization must be found for the output of this merger – that is, a lexical item must be found that can lexicalize it. Lexical items (LIs) are stored links between (maximally) three separate types of information, as shown in (8): the syntactic structure that LI can lexicalize in the form of a stored syntactic tree, a gestural-articulatory representation associated to this lexicalization, and some extra-grammatical conceptual import.

(8) LI_{27}: ⟨ /boʊl/, NP , [BOWL] ⟩

The principle regulating lexicalization (*viz.* the licensing of syntactic structure) is the Matching Condition in (9). When more than one LI match the tree built in the syntactic Workspace, the LI with the least amount of irrelevant features (*viz.* features not contained in the tree involved in matching) wins the competition, a principle known in the literature as Minimize Junk (Starke 2009), which essentially amounts to Kiparsky's (1973) Elsewhere Condition (De Clercq et al. 2025).

(9) (De Clercq et al. 2025: 16)

A lexically stored constituent *L* matches a syntactic phrase *S* iff *S* is identical to *L*.

A derivational cycle therefore involves both structure building and licensing via lexicalization. This procedure builds up structures by cyclically merging new atomic functional features. These features are ordered among themselves by hierarchical relations that define a functional sequence (*fseq*), which is assumed to be universal. Any syntactic derivation draws a selection of features from the *fseq* whose composition yields the grammatical meaning of the structure to be derived, and cyclically merges the selected features one by one in the order dictated by the universal *fseq*.

The operations involved in this process of structure building and lexicalization and their order of application are universal and deterministically defined by the Lexicalization Algorithm (LA). As shown in (10), the LA involves the merger of a new atomic feature, followed by an attempt to

find a matching LI for the resulting structure (step a.). If no match is found, the LA dictates the application of a series of possible repair strategies (steps b. to e.).

(10) MERGE F

a. LEXICALISE [FP].

b. LEXICALISATION-DRIVEN MOVEMENT I: If fail, evacuate the closest labeled non-remnant constituent, re-try a.

c. LEXICALISATION-DRIVEN MOVEMENT II: If fail, evacuate the immediately dominating constituent, re-try a. (recursive)

d. BACKTRACKING: If fail, go back to the previous cycle and try the next option for that cycle, re-try a. (recursive)

e. COMPLEX LEFT BRANCH: if fail, build a new derivation in an auxiliary workspace providing F and merge it with the main derivation from the original workspace, projecting [FP].

As soon as a repair strategy yields a configuration that finds a match in the lexicon, the algorithm stops and the corresponding configuration may be either externalized or be the input of a further derivational cycle. Note that since both the *fseq* and the LA are universal, the only variable in this process is when Matching will yield convergence. That is, any variation in the specific configuration in which a given selection of atomic elements can be combined is ultimately determined by the content of the Lexicon, since Matching involves identity between the structure derived by some step of the LA and some lexically stored tree (Starke 2011; also see Caha 2021 for a discussion of how this perspective on variation relates to the Borer-Chomsky Conjecture). To be clear, what is relevant for this paper is the number of all the possible outputs of the LA, abstracting away from the role of a specific Lexicon in determining when the LA will stop. In other words, the goal is to count all possible output configurations across languages.

4.1.1 Lexicalization-driven movements

Consider the different types of repair strategies in (10). Steps b. and c. trigger the application of the same type of operation, namely evacuation or lexicalization-driven movements of some

subtree. By assumption, lexicalization-driven movement of some subtree merges it at the root without projecting a label and leaves no trace in the extraction site, thus leaving a unary branching node (Caha 2009, 2019, Starke 2018).

As a toy example, consider the LIs in (11), determining the derivation in (12).

(11) LI_{27}: ⟨ /boʊl/, NP , [BOWL] ⟩ LI_{12}: ⟨ /z/, PLP ⟩

Starting from some nominal constituent (here labeled NP for the sake of simplicity) as in (12a), this structure is matched by LI_{27} in (11), as represented graphically by a circle in (12b). Suppose that the structure to be derived is that of a plural nominal and that the next derivational cycle therefore involves the merger of the atomic feature PL, as in (12c). Now, step a. of the LA will fail, since no LI in (11) contains a tree identical to the output of this merger.[5] Therefore, step a. of the LA fails, triggering the application of step b. In this configuration, the labeled non-remnant constituent closest to PlP is NP. Lexicalization-driven movement of NP yields the configuration in (12d), with a root unlabeled node and PL now dominated by a unary node, PlP. In this configuration, PlP is matched by LI_{12}, which contains an identical tree. As a consequence, lexicalization can be achieved by individually matching NP and PlP, as shown in (12e).

(12) a. NP b. NP /boʊl/

c. PLP PL NP d. NP PLP PL e. NP PLP PL /boʊl/ /z/

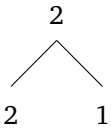
Two properties of lexicalization-driven movements are worth highlighting in this context. The first is that their application at some cycle enables further applications at the next one, adding

[5]Note that the Matching Condition involves identity between phrases, and so it is not enough for an LI to contain the relevant feature: the LI must contain a phrasal node dominating that feature and identical to the corresponding node in the syntactic tree.

up to the total number of configurations that can be derived. This is due to the fact that the first target of lexicalization-driven movement must be a labeled node (step b. in (10)), in combination with the fact that by assumption such movements create an unlabeled root node.

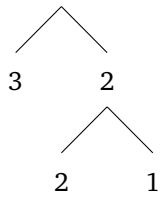
To see how, consider the following abstract derivation. At the first cycle, Merge produces a labeled binary tree with two leaves (13). In this configuration, any further steps in the LA are vacuous, hence this tree is the only possible output.

(13)

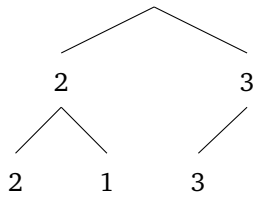


At the next cycle, a new feature is merged. In keeping with the LA, the resulting tree can be lexicalized as is (14a), or after lexicalization-driven movement. This configuration only presents one candidate target for evacuation, namely the closest labeled non-remnant constituent, the node dominating 2 and 1. The resulting configuration is as in (14b). At this stage, I am abstracting away from further steps in the LA, to which I return below.

(14) a.

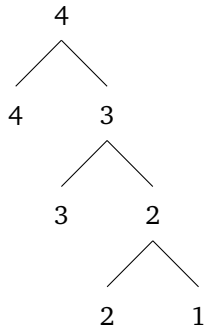


b.

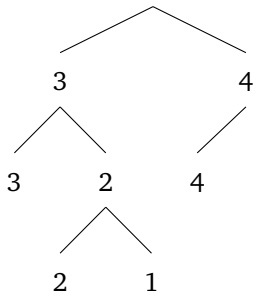


Merger of a fourth feature on top of (14a) yields (15a-b). At the earlier cycle, no lexicalization-driven movement was performed. As a consequence, there are only two possible configurations resulting from this input at this cycle – one resulting from direct lexicalization of the structure (15a), the other from the evacuation of the only candidate for lexicalization-driven movement (15b).

(15) a.

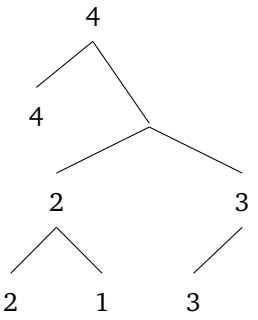


b.

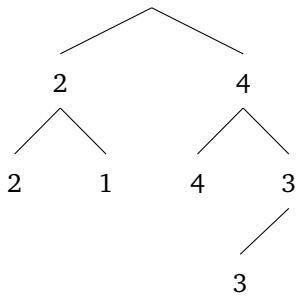


In the other derivational path, an additional possibility is given. Merger of the same feature 4 on top of (14b) results in the configuration in (16a). This can be lexicalized as is or modified via lexicalization-driven movement. The crucial difference is that in this case there are two candidates for movement. Since the evacuation of the binary 'foot' of the structure at the previous stage created an unlabeled node which is now the complement of feature 4, the labeled non-remnant constituent closest to the root is the one labeled by 2. Its lexicalization-driven movement (triggered by step b. of the LA) results in the configuration in (16b). The next step in the LA (step c.) dictates movement of the immediately dominating node, producing (16c).

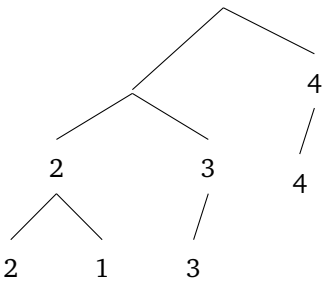
(16) a.



b.



c.



Having illustrated how different options at one cycle determine distinct derivational paths, a second property of lexicalization-driven movements can be highlighted: all possible targets of evacuation in any derivational path were the output of some previous cycle on that same path. That is, lexicalization-driven movements target subtrees that have been the output of a lexicalization cycle, starting from the closest labeled one (Pinzin & Mattiuzzi 2025).

4.1.2 Further steps of the algorithm

Turning to the two further repair strategies encoded in the LA in (10), step d. (BACKTRACKING) is triggered if all evacuation movements available in the configuration derived by the last application of MERGE F failed to yield convergence. Its effect is to allow the derivation to explore alternative paths among those that were in principle available at a previous cycle. It is possible that a given Lexicon allows matching of different configurations among those derivable at a given cycle c_n. At that stage, as soon as the LA yielded a tree that was matched, that option was taken. In case of failure at some later cycle c_{n+x}, BACKTRACKING allows to check whether the alternative lexicalization possibilities available at c_n open a derivational path that eventually leads to convergence at c_{n+x}.[6] In other words, BACKTRACKING triggers the exploration of alternative derivational paths among those made available by the LA. This makes it irrelevant for present purposes, as it does not add to the total number of possible outputs.[7]

The situation is different for Step e. of the LA in (10). This amounts to a Last Resort operation: If none of the previous steps in the algorithm succeeded, it means that none of the possible derivational paths reaching the current cycle produced a configuration that can be matched. That is, all available combinations of operations manipulating the tree in the current derivational Workspace failed to yield convergence. Step e. of the algorithm then dictates the opening of a separate derivation in an independent Workspace, where the MERGE F operation that failed at the last cycle is attempted again. The content of this independent Workspace may be different, as that Workspace might be empty – in which case binary Merge combines F with the empty set – or already contain previously built structure.[8] This means that the configurations that can be obtained via the LA are also different than in the 'main' Workspace, where no convergence was achieved. If a lexicalizable tree containing F can be built in this separate Workspace, the resulting structure is merged at the root of the tree in the 'main' Workspace.

What needs to be highlighted is that the last operation – the merger of the content of the two

[6] Note that since BACKTRACKING applies recursively, it allows to go back multiple cycles in the history of the derivation.

[7] This also means that the claims advanced here also apply under proposals that seek to expunge BACKTRACKING from the LA, as in (Blix 2021) or (van Craenenbroeck 2025)

[8] Alternative views on the starting point of the secondary derivation involve merger of the feature to be lexicalized with the previous feature in the *fseq* (Starke 2018, 2024, De Clercq & Vanden Wyngaerd 2018, Caha 2019, De Clercq 2020). Another open question concerns whether the two cases mentioned above – the 'targeted' creation of a minimal tree containing the feature to be lexicalized vs. lexicalizing it as part of a larger pre-existing structure – have to be distinguished in principle.

independent Workspaces – adds to the number of possible outputs of the LA. More specifically, a derivation running for n cycles in a single Workspace will have a fixed amount of possible outputs. Whenever the trees derived in the 'main' and in the separate Workspaces are combined at some cycle, the number of possible resulting configurations at that cycle corresponds to the product of the number of possible outputs in each Workspace. Moreover, any of the resulting configurations, including those produced by merger of an XP built in a separate Workspace, will be possible inputs of the next derivational cycle, contributing to a faster growth in the number of possible derivational paths.

Quantifying how this interaction between single-Workspace and two-Workspace derivations contributes to the total amount of derivable configurations crucially hinges on a) the possible starting points of a secondary derivation in a separate Workspace, b) the conditions at which a secondary Workspace is closed and its content merged with the tree in the 'main' Workspace, c) whether the merger of a separately built XP results in one or more than one possible configurations.[9] Different views have been expressed in the literature concerning all three points (Starke 2018, De Clercq & Vanden Wyngaerd 2018, Caha 2019, De Clercq 2020, Pinzin & Mattiuzzi 2025), and systematic empirical work to adjudicate among the different possibilities is yet to come.

As is clear, this circumstance represents a limit for the argument proposed here: What can be reliably counted at this stage is not the overall amount of possibilities, but the total number of possible outputs of nanosyntactic derivations running in a single Workspace. At the same time, note that the global, across-Workspace count will necessarily be a function of the local within-Workspace one. That means that pinpointing the details of multiple-Workspace derivations mentioned above will enable to specify the details of this function, but working out the local count remains a necessary step, a point to which I return in the section 5.

The core claim of the paper contributes this necessary first step: the growth of the number of possible outputs of the LA for structures that are built monotonically within a single Workspace is identical to the series of Catalan numbers.

[9] Pinzin & Mattiuzzi (2025) propose a revision of the system that results in both MERGE F and MERGE FP being followed by the same steps of the LA. In the LA in (10), the merger of a phrase via step e. is never followed by further structure-altering operations. In the revised system, any merger of new material (atomic or phrasal) can be followed by lexicalization-driven movements, which means that the number of possible outputs of a derivation combining n atomic features or combining the content of n Workspaces is the same. I return to this point in section 5.

4.2 The LA aligns with the sequence of Catalan numbers

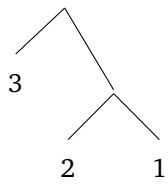
As mentioned above, a standard interpretation of Catalan numbers is that they count the number of possible binary trees with a given number of leaves. To prove the claim advanced here, it is therefore sufficient to show that the LA running for n complete cycles in a single Workspace (*viz.* monotonically building and lexicalizing a structure containing n features) yields all and only the possible binary trees of n leaves.

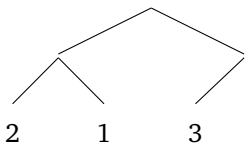
Note that what is ultimately relevant is the identical cardinality of the set of outputs of the LA and the set of binary trees with the same number of leaves. For the sake of this argument, I will therefore abstract away from the distinction between labeled and unlabeled nodes, and focus exclusively on how the geometry of the trees determines a specific distribution of leaves among left and right branches, since each relevant tree uniquely exhibits one such geometry.

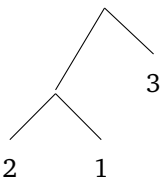
As a first part of the demonstration, consider how all operations in the LA take an input tree and create a new binary branching node at its root. The first operation (step a) does this by Merging a new element at the root. The subsequent operations (steps b-c), involving Lexicalization-driven movement, do it by evacuating some subtree to the root, where crucially evacuation involves the merger of that tree at the root.

As discussed in section 4.1.1, the most common interpretation of evacuation is such that the evacuated object is only represented in this derived position and is erased from the extraction site, turning the node dominating it into a unary-branching node (Starke 2018). From this perspective, lexicalization-driven movements produce trees that are not fully binary. Crucially, however, an abstract transformation can be defined that establishes a one-to-one correspondence between any tree produced by a lexicalization-driven movement and a corresponding fully binary tree with the same number of leaves. This procedure involves replacing all unary nodes in an LA-derived tree with the node they directly dominate.

For instance, from (17a), lexicalization-driven movement produces (17b). Replacing the unary node created by this operation with the node it immediately dominates produces the binary tree in (17c).

(17) a. 

b. 

c. 

Each application of an evacuation movement creates one new unary node, and so for each tree T that is the output of an evacuation movement there exists exactly one corresponding binary tree B such that B is obtained by replacing the unary nodes in T with the node they immediately dominate. The general consequence is that the application of any operation in the LA yields objects that are either binary trees or have a unique corresponding binary tree.

The second part of the demonstration amounts to showing that for any derivational cycle, the set L of objects that are the output of the LA at that cycle is such that all binary trees with the same amount of leaves has a corresponding object in L.

What is crucial here is the recursive application of the LA: Any output of the LA at some cycle is the input of the LA at the next cycle. As discussed in section 4.1.1, all outputs of the LA at some cycle n have two possible continuations at cycle $n+1$, one resulting from the simple merger of a new object X, the other produced by merger of X followed by the evacuation of its whole complement. In addition, the definition of evacuation movements is such that the application of one such movement at cycle n yields further possible outputs at cycle $n + 1$, since on top of the two possibilities just mentioned, it will be possible to evacuate again the same constituent and (in later steps of the LA) any node dominating it (see above).

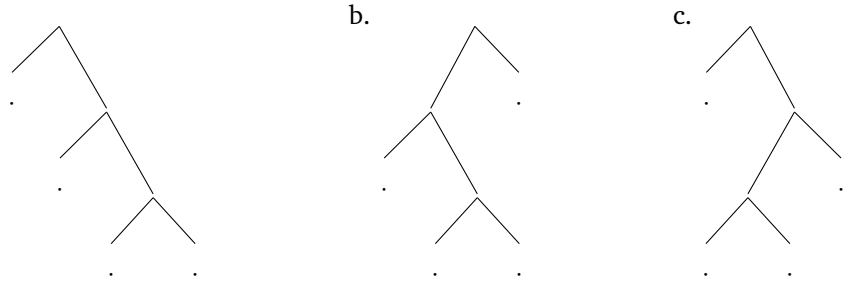
Importantly, lexicalization-driven movements can only target subtrees that were the output of some previous lexicalization cycle (Pinzin & Mattiuzzi 2025), and any derivation ultimately sets off from the most minimal binary tree with two leaves. In combination with the recursive nature of the LA, this means that any possible subtree can be the target of lexicalization-driven movement in some possible derivation.

Taking stock, all possible outputs of the LA have exactly one corresponding binary tree and can be the target of lexicalization-driven movement in some derivation. Any such movement yields an output tree that can itself be uniquely mapped into a binary tree, which has the consequence that the combination of all structural manipulations resulting from the application of the LA yields a set of objects that uniquely correspond to all and only the possible binary-tree geometries. Since

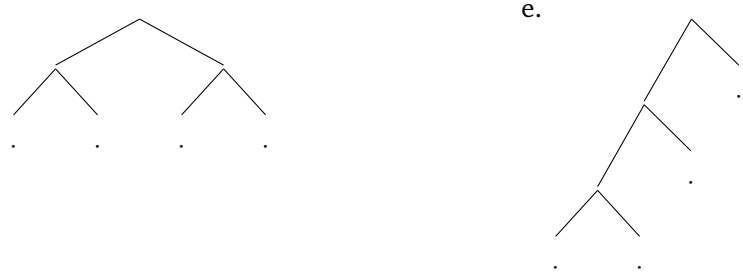
the latter are counted by the Catalan numbers, this one-to-one mapping warrants the conclusion that the sum of the total possible outputs of the LA for n cycles also grows with the Catalan numbers, and amounts to C_{n-1}, as claimed above. [10]

To illustrate, there are $C_3 = 5$ possible geometries for binary trees of 4 leaves. For the sake of the discussion, let us ignore the identity of the leaves, and instead focus on how the geometry of each tree corresponds to a different way of grouping leaves under left and right branches. The tree in (18a) has a rigid right-branching structure; (18b) has mixed branching with one leaf on the right branch and three on the left; (18c) is its mirror image – also with mixed branching, but with one leaf on the left branch and three on the right; (18d) has a symmetrical structure with two leaves per branch; finally, (18e) is strictly left-branching.

(18) a. b. c.



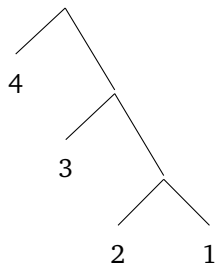
d. e.



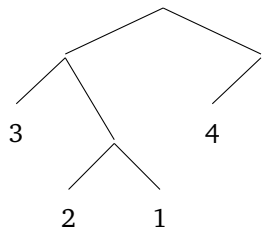
Consider now the 5 possible outputs of the LA for a structure containing 4 objects in (19), a schematic representation of the structures whose step-by-step derivation I discussed in section 4.1.1 above.

[10] A Jupiter Notebook allowing a dynamic visualization of the growth of the two functions and the code it requires are available on GitHub (tommattiuzzi/nanosyntactic-lexicalization-algoritm). The repository contains a Python-based implementation of the LA that is the result of joint work with Francesco Pinzin.

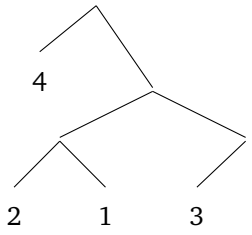
(19) a.



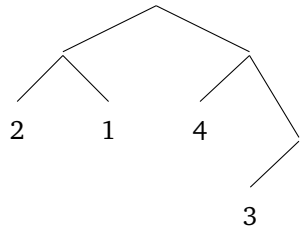
b.



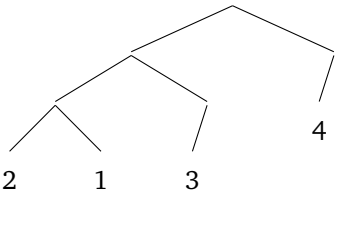
c.



d.

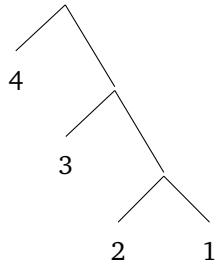


e.

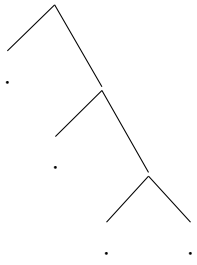


Note how the five trees fulfill the same distribution of leaves on left and right branches as above: strict right-branching in (19a), mixed branching with three leaves on the left branch and one on the right branch in (19b), mixed branching with one leaf on the left branch and three on the right one in (19c), two nodes per branch in (19d), and (abstracting away from unary nodes) strict left branching in (19e). By the transformation that simplifies unary nodes, it is therefore possible to establish the correspondences in (20).

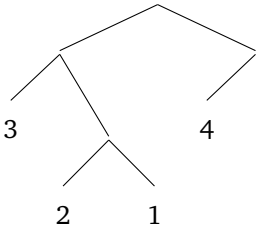
(20) a.



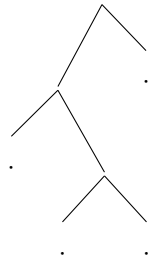
↔

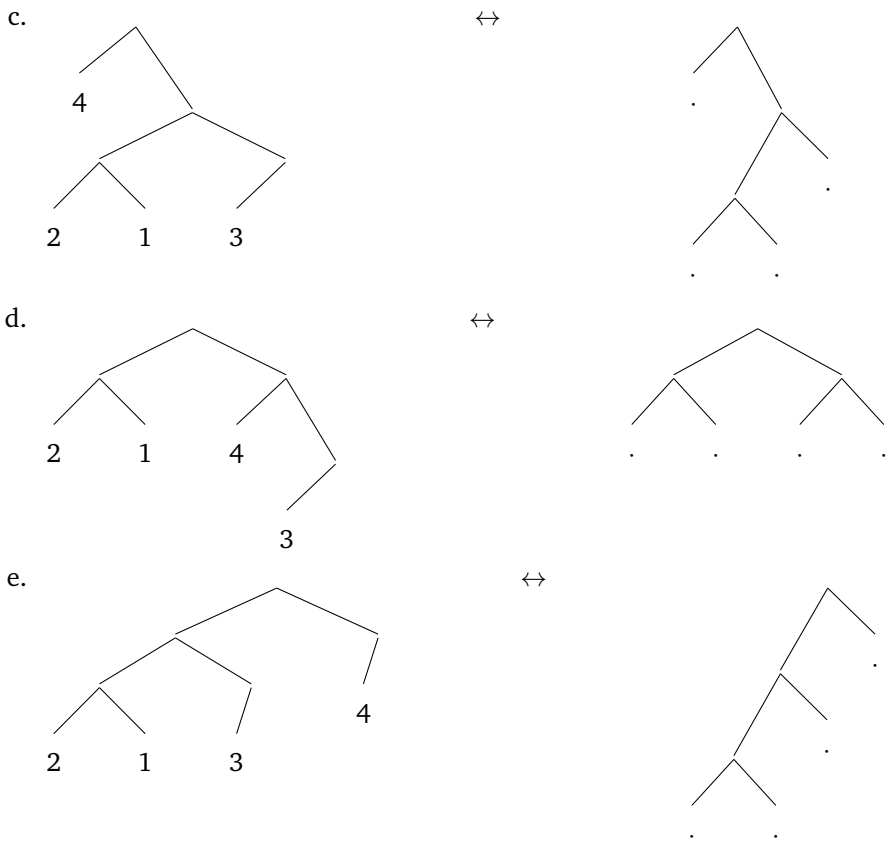


b.



↔





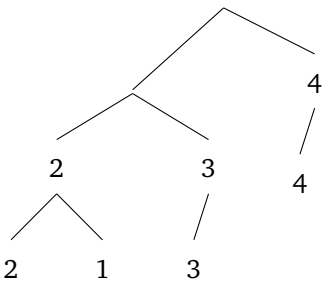
5 Discussion and open issues

What does this demonstration give us with respect to the question introduced above about the theoretical boundaries of syntactic variation? Assuming a nanosyntactic theory of structure building and externalization, we can identify the series of Catalan numbers as a necessary component of the theoretical limit on morphosyntactic variability.

To be clear, Catalan numbers cannot be directly identified as the quantitative boundary of variation. On one hand, the total possible outputs of the LA are larger than the Catalan numbers once derivations involving multiple workspaces are considered, as discussed in section 4.1.2. On the other, directly observable linguistic objects result from how the output tree of a derivation is lexicalized. Any structure where not all phrasal nodes are in a direct domination relation is amenable to different lexicalization 'strategies' – for instance, the structure in (21) may be lexicalized by a single LI matching its root node, by two LIs separately matching the left and the

right branch, or by three LIs, each matching one labeled node, as shown below.

(21)



This means that abstracting away from phonological differences, the number of configuration-lexicalization pairs is higher than the boundary set by the Catalan numbers.

What is relevant is that in both cases the Catalan numbers represent the necessary basis on which the total number of possibilities must be computed. When considering derivations involving multiple Workspaces, the possibilities available in each Workspace compound, jointly determining the total number of theoretically defined outputs. Similarly, the possible shapes of lexically-stored trees determining observable patterns are bound by the configurations that can be derived, and so counting the latter contributes to the definition of the possible shapes and sizes of LIs, and their combination.

To be concrete, consider how the interaction of multiple Workspaces expands the number of possible outputs. As I discussed above, exactly how the space of possibilities expands hinges on the specific details of the derivational scenarios that are ruled in. To discuss one example, assume that when a secondary workspace is opened to build a minimal binary tree containing the feature to be lexicalized, the resulting tree is lexicalized in that Workspace and then immediately merged back with the content of the main Workspace, closing the secondary one (see Starke 2018, Caha 2019 for discussion), as dictated by the COMPLEX LEFT BRANCH step of the algorithm in 10, repeated below for reference.

(10e) COMPLEX LEFT BRANCH: if fail, build a new derivation in an auxiliary workspace providing F and merge it with the main derivation from the original workspace, projecting [FP].

Let us also make the simplifying assumption that this only concerns empty secondary Workspaces, that is, cases where a feature to be lexicalized F is merged into an empty Workspace, followed

by the immediate merger of the result to the main tree. This prevents any Complex Left Branch from growing beyond one derivational cycle, an assumption that I am only making here for the sake of the exposition.

Starting from the second derivational cycle[11], this adds possible derivational outputs on top of those derivable via lexicalization-driven movements. In turn, these outputs form legitimate inputs of subsequent cycles, which may themselves involve any applicable operations among those dictated by the LA. Moreover, building and merging a new Complex Left Branch is an option at any cycle, determining $C_1 = 1$ additional output at that cycle for each output tree of the previous cycle, since we are only considering derivations running for exactly one cycle in the secondary Workspace. That is, each application of Complex Left Branch effectively opens a new set of derivational paths, whose initial cardinality is given by the number of possible trees to which the new Complex Left Branch can be merged. This additional set of derivational paths grows itself with the Catalan series, while also providing additional possible inputs for further additions of a Complex Left Branch at later cycles. To illustrate the recursive compounding of possibilities opened up by the addition of Complex Left Branches, consider how the total number of outputs can be expressed in terms of sums for the first derivational steps (22).

(22)
$$T_1 = C_1 = 1$$
$$T_2 = C_2 + C_1(C_1) = 2 + 1(1) = 3$$
$$T_3 = C_3 + C_2(C_1) + C_1(C_2 + C_1(C_1)) = 5 + 2(1) + 1(3) = 10$$
$$T_4 = C_4 + C_3(C_1) + C_2(C_2 + C_1(C_1)) + C_1(C_3 + C_2(C_1) + C_1(C_2 + C_1(C_1))) = 14 + 5(1) + 2(3) + 1(10) = 35$$
$$\ldots$$

At each cycle, the application of Complex Left Branch determines one more term in the sum of total outputs. This term is the product of a factor that grows with the Catalan numbers at subsequent cycles in the derivation and a constant that 'records' the number of trees available when the merger of the Complex Left Branch applied. Generalizing, the total number of outputs T_m at a given derivational cycle m is given by summing the corresponding Catalan number C_m (the number of outputs of the single-Workspace derivation) with the sum of all the Catalan-

[11] At the first cycle, a basic binary tree is built, and no repair strategies are available, see 4.1.

bound derivations that resulted from applications of COMPLEX LEFT BRANCH at any step in the derivation, including the current one:

(23)
$$T_m = C_m + \sum_{j=1}^{m-1} C_j(T_{m-j})$$

As it turns out, this recursive formula can be expressed in closed form as a function of the m-th Catalan number (see the demonstration in Appendix A):

(24)
$$T_m = \frac{m+1}{2} C_m$$

Under the specific (simplifying) assumptions adopted, the number of possible outputs of the algorithm with the addition of (bit-sized) Complex Left Branches is larger than the one for single-Workspace derivations by a factor of $\frac{m+1}{2}$. Different assumptions about how multiple Workspaces might grow in parallel and interact will entail different values of the factor, without changing the basic logic of characterizing the space of possibilities in terms of recursive sums of Catalan numbers.

Summarizing, we can conclude that both the number of possible outputs of multi-Workspace derivations and the total number of possible lexicalizations of structures with a given number of features are a function of the Catalan numbers. Future work addressing the specifics of how multiple Workspaces are combined will make it possible to work out these two functions and achieve a mathematical characterization of the theoretical limit on possible morphosyntactic variation.

Concluding on a more speculative note, another aspect of the 'scaling up' of the points made in this paper is worth highlighting. What I showed above is that nanosyntactic derivations are 'Catalan-bound' at the smallest possible scale, the level of individual Workspaces where individual atomic features are assembled in trees. Under the partial revision of the LA proposed by Pinzin & Mattiuzzi (2025), the same set of operations can apply after any structure-building operation, as per the revised lexicalization algorithm in (25). Under this view, lexicalization-driven movements can be triggered after the merger of an atomic feature (MERGE F) as well as of a phrase (MERGE FP).[12]

[12]Strictly speaking, independent reasons always yield convergence after MERGE FP before theBACKTRACKING step is

(25) Lexicalisation Algorithm:

1. MERGE F	a. LEXICALISE [FP].
2. MERGE FP	b. LEX-DRIVEN MOV I: If fail, evacuate the closest labelled non-remnant constituent, re-try a.
	c. LEX-DRIVEN MOV II: If fail, evacuate the immediately dominating constituent, re-try a. (recursive)
	d. BACKTRACKING: If fail, go back to the previous cycle and try the next option for that cycle, re-try a. (recursive)

Note that since the same operations are involved, the number of possible paths for a derivation where Merge only combines trees built in independent Workspaces is given by the Catalan numbers, exactly like for single-Workspace derivations. This means that the possible derivational paths only exceed the boundary set by Catalan numbers is when considering the intertwining of MERGE F and MERGE FP. Derivations that build structure and lexicalize it exclusively via MERGE F operate at the smallest possible scale, where atomic elements are assembled. Those that instead only combined pre-built phrases via MERGE FP operate at the largest possible scale, since however big two structures, their merger amounts to the merger of the content of the two Workspaces in which they were derived. What this point indicates is that the growth of the derivation at the smallest and the largest possible scale is the same and is given by the Catalan numbers. This boundary is only exceeded when considering derivations that intertwine MERGE F and MERGE FP. As mentioned above, more work is needed on the theoretical side to be able to assess exactly the growth of the space of possibilities in this latter case, which however will necessarily be again a function of the Catalan numbers.

A Total outputs with bit-sized Complex Left Branches

The formula in (24), repeated here, specifies the recursive procedure to count the total number of output trees T_m at the m-th cycle of a derivation where trees can be built and lexicalized in a single Workspace or by directly combining the output of secondary workspaces, as described in

reached. To the extent that this does not affect the total number of possible output configurations (see 4.1.2), this difference is irrelevant for present purposed.

Section 5.

(24)

$$T_m = C_m + \sum_{j=1}^{m-1} C_j T_{m-j}$$

Setting $C_0 = 0$ and $T_0 = 0$, both sequences can be represented by their ordinary generating functions (Wilf 2005):

$$C(x) = \sum_{m=1}^{\infty} C_m x^m \quad \text{and} \quad T(x) = \sum_{m=1}^{\infty} T_m x^m$$

The summation term $\sum_{j=1}^{m-1} C_j T_{m-j}$ represents the coefficient of x^m in the Cauchy product (convolution) of $C(x)$ and $T(x)$ (Wilf 2005). Because $C_0 = T_0 = 0$, the $j = 0$ and $j = m$ terms evaluate to zero:

$$\sum_{j=0}^{m} C_j T_{m-j} = C_0 T_m + \sum_{j=1}^{m-1} C_j T_{m-j} + C_m T_0 = \sum_{j=1}^{m-1} C_j T_{m-j}$$

Multiplying both sides of (24) by x^m and summing over all $m \geq 1$ yields:

$$\sum_{m=1}^{\infty} T_m x^m = \sum_{m=1}^{\infty} C_m x^m + \sum_{m=1}^{\infty} \left(\sum_{j=1}^{m-1} C_j T_{m-j} \right) x^m$$

In terms of generating functions, this is equivalent to:

$$T(x) = C(x) + C(x)T(x)$$

Solving for $T(x)$ expresses the generating function for total outputs T in terms of $C(x)$:

$$T(x) = \frac{C(x)}{1 - C(x)}$$

Now, the ordinary generating function for the standard Catalan sequence (Stanley 2015) $c(x)$ is:

$$c(x) = \sum_{n=0}^{\infty} c_n x^n = \frac{1 - \sqrt{1 - 4x}}{2x}$$

Since we are excluding from the count cycle 0 of the derivation ($C_0 = 0$), $C(x)$ corresponds to the Catalan series shifted by subtracting $c_0 = 1$:[13]

$$C(x) = c(x) - 1 = \frac{1 - \sqrt{1 - 4x}}{2x} - 1$$

Setting $y = \sqrt{1 - 4x}$ (which implies $2x = \frac{1-y^2}{2}$), $C(x)$ simplifies to:

$$C(x) = \frac{1 - y}{\frac{1-y^2}{2}} - 1 = \frac{2(1 - y)}{(1 - y)(1 + y)} - 1 = \frac{2}{1 + y} - 1 = \frac{1 - y}{1 + y}$$

Substituting $C(x) = \frac{1-y}{1+y}$ into the equation for $T(x)$ gives:

$$T(x) = \frac{\frac{1-y}{1+y}}{1 - \frac{1-y}{1+y}} = \frac{1 - y}{2y}$$

Substituting $y = \sqrt{1 - 4x}$ back into $T(x)$ yields:

$$T(x) = \frac{1 - \sqrt{1 - 4x}}{2\sqrt{1 - 4x}} = \frac{1}{2\sqrt{1 - 4x}} - \frac{1}{2}$$

Using the standard binomial series expansion for central binomial coefficients (Wilf 2005),

$$\frac{1}{\sqrt{1 - 4x}} = \sum_{m=0}^{\infty} \binom{2m}{m} x^m = 1 + \sum_{m=1}^{\infty} \binom{2m}{m} x^m$$

we obtain:

$$T(x) = \frac{1}{2}\left(1 + \sum_{m=1}^{\infty} \binom{2m}{m} x^m\right) - \frac{1}{2} = \sum_{m=1}^{\infty} \left[\frac{1}{2}\binom{2m}{m}\right] x^m$$

Since we know that $T(x) = \sum_{m=1}^{\infty} T_m x^m$, we can equate the coefficients of x^m for $m \geq 1$, which gives:

$$T_m = \frac{1}{2}\binom{2m}{m}$$

Finally, expressing Catalan numbers in terms of binomial coefficients ($C_m = \frac{1}{m+1}\binom{2m}{m}$), we

[13] That is, the sequence starts as $c_0 = 1, c_1 = 1, c_2 = 2, , c_3 = 5, ...,$. In using it as a way to count the outputs of the derivation, we are practically disregarding $c_0 = 1$, but this is indeed part of the sequence and hence we must apply this correction when plugging in the generating function $c(x)$ for $C(x)$ in the equation, where $C(x)$ corresponds to the sequence of counts, i.e. $C_1 = 1, C_2 = 2, C_3 = 5, ...$

can express T_m directly as a function of C_m:

$$T_m = \frac{1}{2}\binom{2m}{m} = \frac{m+1}{2}\frac{1}{m+1}\binom{2m}{m} = \frac{m+1}{2}C_m$$

B The constraining effect of the *fseq*

What if we didn't assume a *fseq*? How would that affect the number of possibilities?

After all, binary Merge is in itself blind to any property ordering its input. For this reason, two opposite options are imaginable. One is to take Merge and Narrow Syntax to build binary trees from the input without restrictions, with improper output structures being filtered out at the interface with the Sensory-Motor and the Conceptual-Intentional systems (Chomsky 2013, 2015, Ginsburg 2024). The opposite view assumes the input of Merge to be subject to independent restrictions that determine or restrict the order in which the atoms of the structure-building computation are combined. The latter approach has been pursued to varying degrees of granularity in the make-up of syntactic structures, ranging from a simple C-T-v skeleton to cartographic hierarchies (Cinque & Rizzi 2010, Rizzi 1997, Cinque 1999) to the sub-word level of functional sequences studied in Nanosyntax (Caha 2009, Starke 2009, De Clercq et al. 2025). Somewhat simplifying the picture, let us compare two abstract options, one that assumes no restrictions on the order in which syntactic atoms are combined by Merge, and the opposite one in which a total relation exists among the atoms that determines their order of Merge, like the functional sequence (*fseq*) assumed by Cartography or Nanosyntax. This comparison highlights the crucial role played by the *fseq*: If the structure-building procedure via binary Merge is not based on a *fseq*, the actual number of possibilities defined grows faster than the series of Catalan number.

To see how, consider a structure-building derivation that takes as input an unordered set of atoms, call it the Numeration. For a Numeration of n elements, there are $n!$ ways in which these elements can be ordered. Therefore, if nothing independently specifies the order in which the elements in the Numeration are to be merged, there are $n!$ different possible sequences in which the elements in the Numeration can be fed to the structure-building procedure. For instance, a Numeration of 4 atoms defines the 24 possibilities shown in (26a).

(26) a. Numeration: {1, 2, 3, 4}

b. Possible orders of Merge:

$< 1, 2, 3, 4 >$	$< 2, 1, 3, 4 >$	$< 3, 1, 2, 4 >$	$< 4, 1, 2, 3 >$
$< 1, 2, 4, 3 >$	$< 2, 1, 4, 3 >$	$< 3, 1, 4, 2 >$	$< 4, 1, 3, 2 >$
$< 1, 3, 2, 4 >$	$< 2, 3, 1, 4 >$	$< 3, 2, 1, 4 >$	$< 4, 2, 1, 3 >$
$< 1, 3, 4, 2 >$	$< 2, 3, 4, 1 >$	$< 3, 2, 4, 1 >$	$< 4, 2, 3, 1 >$
$< 1, 4, 2, 3 >$	$< 2, 4, 1, 3 >$	$< 3, 4, 1, 2 >$	$< 4, 3, 1, 2 >$
$< 1, 4, 3, 2 >$	$< 2, 4, 3, 1 >$	$< 3, 4, 2, 1 >$	$< 4, 3, 2, 1 >$

For each individual n-sequence, the number of possible output trees is C_{n-1}, like in () above. If the output structures are assumed to be ordered trees, where left and right branches are distinguished, the total number of possibilities is much higher, since the value of $C_n - 1$ must be multiplied by the number of all possible merge-order sequences, $n!$, as shown in (27).

(27)

$$N_{nofseq,ordered} = C_{n-1}\, n!$$

If instead trees are assumed to be unordered and symmetric structures are not distinguished, the total number of possibilities is lower, but still higher than C_{n-1}. Since each binary tree of n leaves has $n - 1$ non-terminal nodes and each non-terminal node is a point distinguishing a left and a right branch, any unordered binary tree of n leaves has 2^{n-1} ordered counterparts. This means that when considering the number of possible output unordered trees of a binary structure-building derivation in the absence of a *fseq*, the total number must be reduced by a factor of 2^{n-1}, as in (28).

(28)

$$N_{nofseq,unordered} = C_{n-1}\frac{n!}{2^{n-1}}$$

The conclusion to draw from present purposes is that the series of Catalan numbers does define a theoretical limit on the number of binary trees that can be built from a set of input atoms *iff* there is a fixed sequence in which such atoms are added, that is a total *fseq*. Loosening this constraint on the possible order of Merge of syntactic atoms determines a growth in the number

of possibilities defined. As a visual reference, Figure 2 shows how the number of possibilities grows for numerations of increasing size under the two boundary cases discussed, namely in the absence of any restrictions on the order of Merge or assuming the order to be uniquely determined by a *fseq*. Lifting the constraint imposed by a fixed *fseq* has the consequence that the absolute number of possibilities rapidly diverges by several orders of magnitude from the series of Catalan numbers, even under the conservative assumption that symmetric trees are not distinguished.

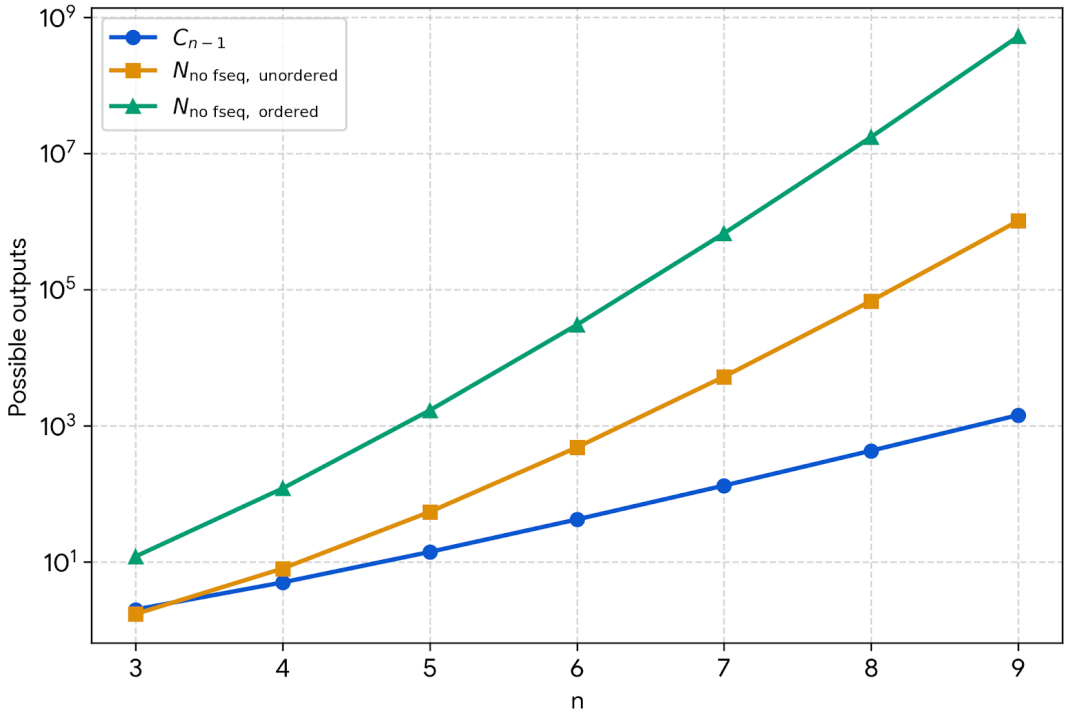


Figure 2: Log-scaled growth rate of the number of possible (ordered or unordered) binary trees of n built by binary Merge on n syntactic atoms, with and without a *fseq*.

References

Abels, Klaus & Neeleman, Ad (2012). Linear Asymmetries and the LCA. *Syntax (Oxford, England)* 15.1, pp. 25–74. https://doi.org/10.1111/j.1467-9612.2011.00163.x.

Baunaz, Lena & Lander, Eric (2018). Nanosyntax: The basics. In: *Exploring nanosyntax*. Baunaz, Lena & Haegeman, Liliane & De Clercq, Karen & Lander, Eric (eds.) Oxford University Press, pp. 3–56.

Blix, Hagen (2021). Phrasal spellout and partial overwrite: On an alternative to backtracking. *Glossa: a journal of general linguistics* 6.1.
https://doi.org/10.5334/gjgl.1614.

Bobaljik, Jonathan David (2017). "Distributed Morphology". In: *Oxford Research Encyclopedia of Linguistics*: Oxford University Press.
https://doi.org/10.1093/acrefore/9780199384655.013.131.

Boeckx, Cedric (2011). Approaching parameters from below. In: *The biolinguistic enterprise: New perspectives on the evolution and nature of the human language faculty*. Di Sciullo, Anna Maria & Boeckx, Cedric (eds.) Oxford: Oxford University Press, pp. 205–221.

Boeckx, Cedric (2016). Considerations pertaining to the nature of logodiversity. In: *Rethinking Parameters*. Eguren, Luis & Fernandez-Soriano, Olga & Mendikoetxea, Amaya (eds.) 1st ed.: Oxford University PressNew York, pp. 64–104.
https://doi.org/10.1093/acprof:oso/9780190461737.003.0003.

Caha, Pavel (2009). *The nanosyntax of case*. PhD dissertation. Universitetet i Tromsø.

Caha, Pavel (2019). *Case competition in Nanosyntax. A study of numeral phrases in Ossetic and Russian*: lingbuzz/004875.

Caha, Pavel (2021). Minimalism and Nanosyntax. LingBuzz Published In: to appear in Grohmann, Kleanthes & Evelina Leivada (eds.): The Cambridge Handbook of Minimalism, Cambridge University Press.

Caha, Pavel (2022). The layered structure of concord: A Nanosyntactic approach. LingBuzz Published In:

Chomsky, Noam (1995). *The Minimalist program*. Current studies in linguistics 28. Number: 28. Cambridge, Mass: The MIT Press. 420 pp.

Chomsky, Noam (2005). Three Factors in Language Design. *Linguistic Inquiry* 36.1, pp. 1–22.

Chomsky, Noam (2013). Problems of projection. *Lingua. International review of general linguistics. Revue internationale de linguistique générale* 130, pp. 33–49.

Chomsky, Noam (2015). Problems of projection: Extensions. In: *Linguistik Aktuell/Linguistics Today*. Di Domenico, Elisa & Hamann, Cornelia & Matteini, Simona (eds.) Vol. 223. Amsterdam: John Benjamins Publishing Company, pp. 1–16.
https://doi.org/10.1075/la.223.01cho.

Cinque, Guglielmo (1999). *Adverbs and functional heads: A cross-linguistic perspective*: Oxford University Press.

Cinque, Guglielmo (2005). Deriving Greenberg's Universal 20 and its exceptions. *Linguistic inquiry* 36.3, pp. 315–332.

Cinque, Guglielmo (2025). The superiority of a movement approach to Greenberg's Universal 20 (a reply to Dryer 2018). *Syntactic Theory and Research* 1.1, p. 17346.
https://doi.org/10.16995/star.17346.

Cinque, Guglielmo & Rizzi, Luigi (2010). *The Cartography of Syntactic Structures*: Oxford University Press.
https://doi.org/10.1093/oxfordhb/9780199544004.013.0003.

Collins, Chris (2022). The complexity of trees, universal grammar and economy conditions. *Biolinguistics* 16, e9573.
https://doi.org/10.5964/bioling.9573.

De Clercq, Karen (2020). *The Morphosyntax of Negative Markers: A Nanosyntactic Account*: De Gruyter.
https://doi.org/10.1515/9781501513756.

De Clercq, Karen & Caha, Pavel & Starke, Michal & Vanden Wyngaerd, Guido (2025). Nanosyntax: State of the art and recent developments. In: *Nanosyntax and the Lexicalization Algorithm*. Caha, Pavel & De Clercq, Karen & Vanden Wyngaerd, Guido (eds.) 1st ed.: Oxford University PressOxford, pp. 1–44.
https://doi.org/10.1093/9780198947158.003.0001.

De Clercq, Karen & Vanden Wyngaerd, Guido (2018). Unmerging analytic comparatives. *Jezikoslovlje*.

Ginsburg, Jason (2024). Constraining Free Merge. *Biolinguistics* 18, e14015.
https://doi.org/10.5964/bioling.14015.

Haegeman, Liliane M. V. (1994). *Introduction to government and binding theory*. Blackwell textbooks in linguistics 1. Malden, Mass.: Blackwell. 701 pp.

Halle, Morris & Marantz, Alec (1993). Distributed morphology and the pieces of inflection. In: *The view from Building 20*. Hale, Kenneth & Keyser, Samuel Jay (eds.) Cambridge, Massachusetts: MIT Press, pp. 111–176.

Halle, Morris & Marantz, Alec (1994). Some key features of Distributed Morphology. *MIT working papers in linguistics* 21.275.

Kayne, Richard S. (1984). *Connectedness and Binary Branching*. Dordrecht: Foris Publications.

Kiparsky, Paul (1973). Elsewhere in phonology. In: *A Festschrift for Morris Halle*. Anderson, Stephen & Kiparsky, Paul (eds.) New York: Holt, Rinehart & Winston, pp. 93–106.

Martin, Alexander & Adger, David & Abels, Klaus & Kanampiu, Patrick & Culbertson, Jennifer (2024). A Universal Cognitive Bias in Word Order: Evidence From Speakers Whose Language Goes Against It. *Psychological Science* 35.3, pp. 304–311. https://doi.org/10.1177/09567976231222836.

Pinzin, Francesco & Mattiuzzi, Tommaso (2025). Word-order information in the Lexicon. *Glossa: a journal of general linguistics* 10.1, p. 18521. https://doi.org/10.16995/glossa.18521.

Rizzi, Luigi (1997). The fine structure of the left periphery. In: *Elements of grammar*: Springer, pp. 281–337.

Stanley, Richard P. (2015). *Catalan Numbers*. 1st ed.: Cambridge University Press. https://doi.org/10.1017/CBO9781139871495.

Starke, Michal (2009). “Nanosyntax: A short primer to a new approach to language, ms.”

Starke, Michal (2011). Towards an elegant solution to language variation: Variation reduces to the size of lexically stored trees. *Ms., Tromsø University*.

Starke, Michal (2018). Complex left branches, spellout, and prefixes. In: *Exploring nanosyntax*. Baunaz, Lena & Haegeman, Liliane & De Clercq, Karen & Lander, Eric (eds.) Oxford University Press, pp. 239–249.

Starke, Michal (2024). “Nanoseminar. Online lecture series at Masaryk university.” Brno.

Van Craenenbroeck, Jeroen (2025). “Blixemes, pointers, and *ABA”.

Wilf, Herbert S. (2005). *generatingfunctionology: Third Edition*. 0th ed.: A K Peters/CRC Press. https://doi.org/10.1201/b10576.